

Review: Bitwise operators

⌘ Bitwise operators

- ✧ Complement (~)
- ✧ AND (&)
- ✧ XOR (^)
- ✧ OR (|)

⌘ Shift operators

- ✧ Left shift ($\text{expr1} \ll \text{expr2}$)
- ✧ Right shift ($\text{expr1} \gg \text{expr2}$)



Ex: Missing number

- Given an array containing n distinct numbers from 0 to n , find the missing number. Implement this using bitwise XOR operation.

- Suppose you have an array
 $\text{arr}[] = [0, 1, 3, 4, 5]$

The missing number is 2.

Ex: Missing number

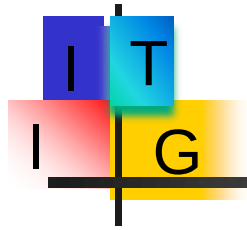
- Given an array containing n distinct numbers from 0 to n , find the missing number. Implement this using bitwise XOR operation.

Example: `arr[] = [0, 1, 3, 4, 5]`

Steps

- XOR all the elements from 0 to 5
- XOR the array elements
- XOR `xor_all` and `xor_arr` to find the missing number

Any number XORed with itself is 0 (e.g., $5 \wedge 5 = 0$).
Any number XORed with 0 is itself (e.g., $5 \wedge 0 = 5$).



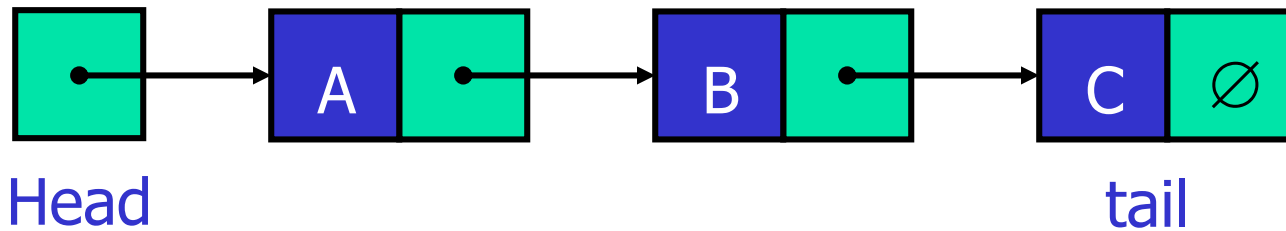
Linked List



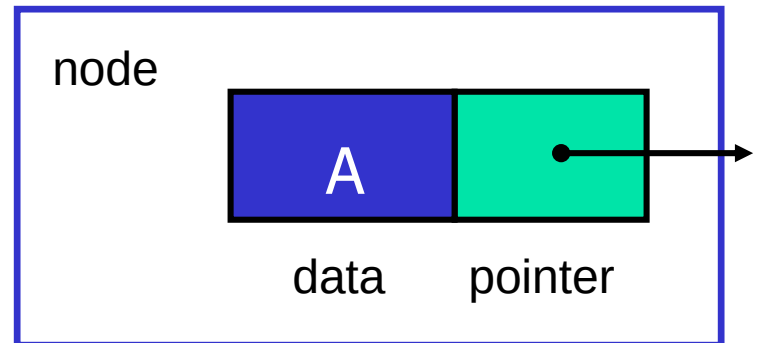
Static vs. dynamic data structure

- Data structure is a way of organizing and storing data efficiently
 - Allow efficient operations (insertion, deletion)
- Static data structure
 - Content can be modified but size (i.e. memory allocated) cannot be changed
- Dynamic data structure
 - Content as well as size of the structure can be modified

What is a linked list



- Linear data structure consisting of a series of connected **nodes**
- Node consists of
 - data (any type)
 - Pointer to next node
- Head points to first node, tail points to null

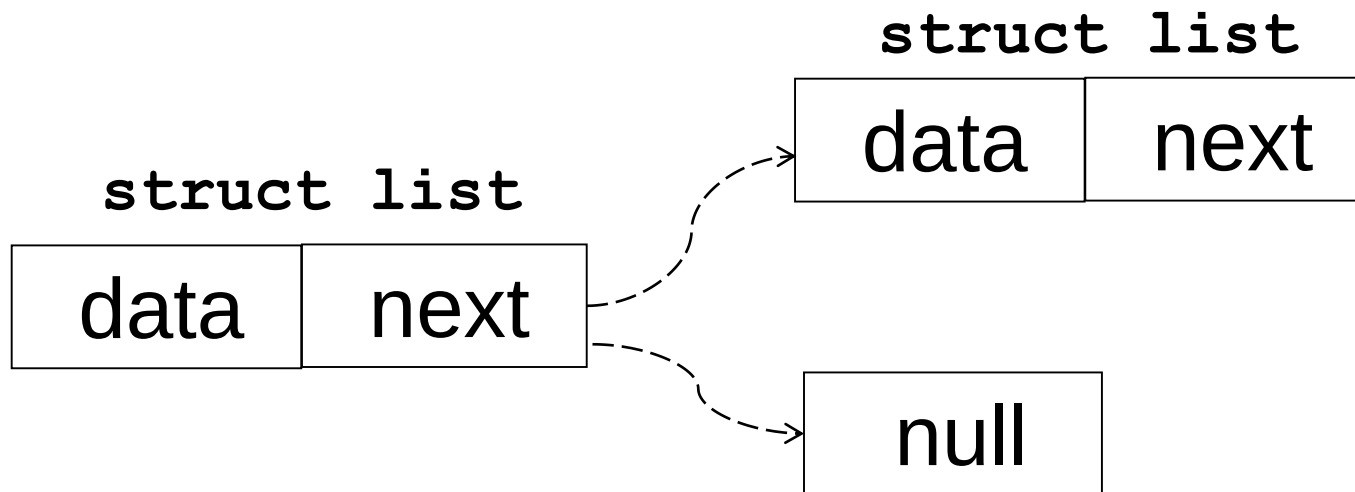




Creating linked list of node (1)

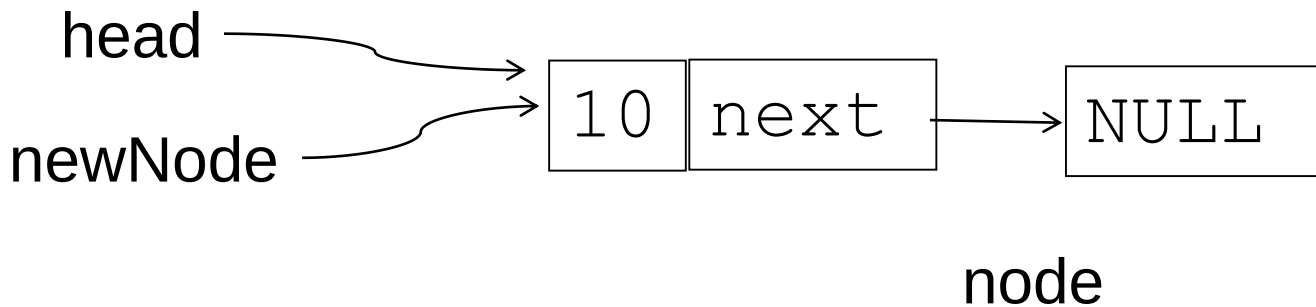
- Step 1: Create a self-referential structure
 - A structure with a member field that points to the same structure type. (Node)
 - Remember a linked list has a head, that points to the first node
 - Create two variables, head and newNode of type Node.
 - Set head to NULL

- 
- The pointer variable `next` can either point to another structure `node` or to `NULL`



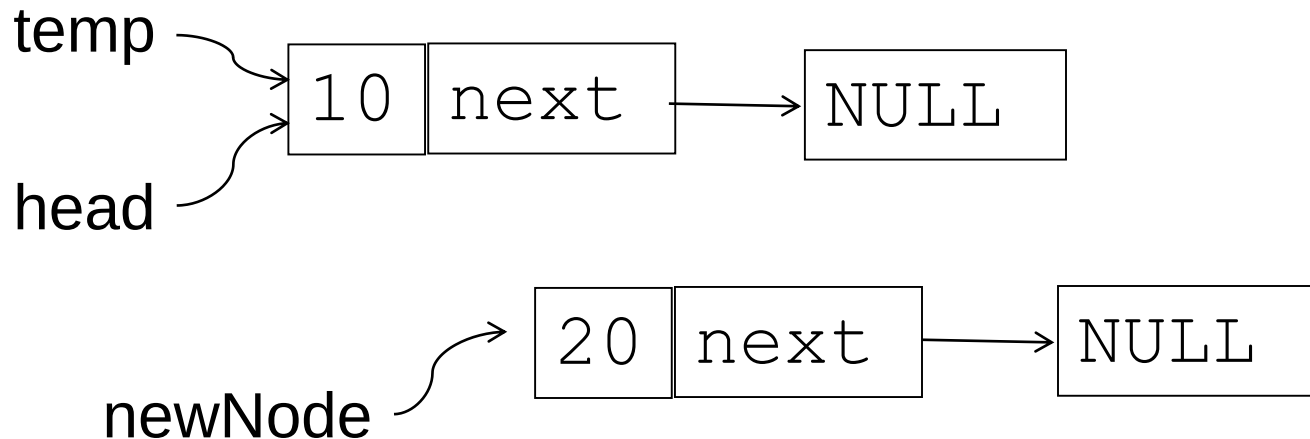
Creating linked list of 1 node (2)

- Step 2: Create a node and assign data
 - Allocate space to newNode
 - Assign data to newNode
 - Set newNode->next to NULL
 - Set head = newNode if head == NULL



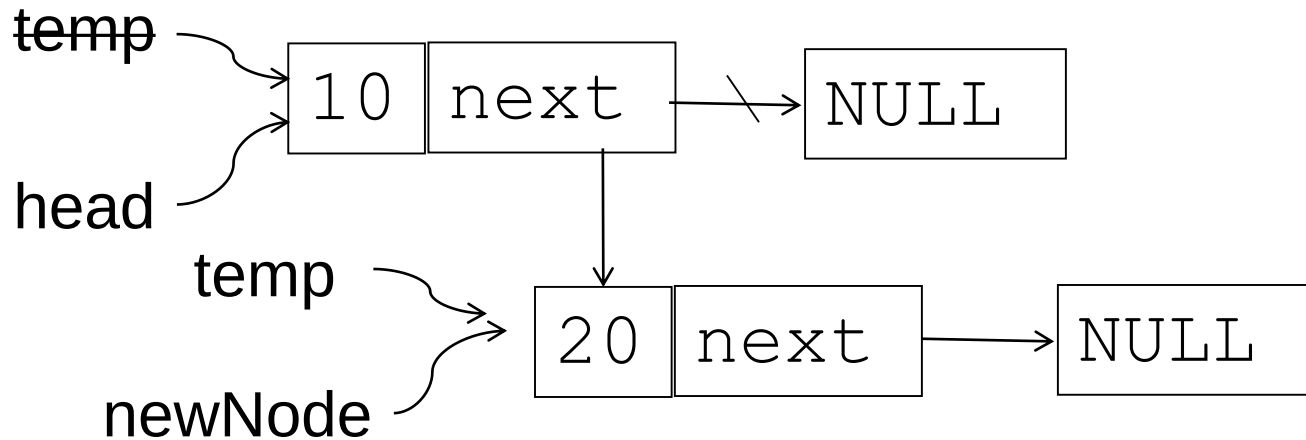
Creating linked list of 2 node

- Step 3: Create the second, ... node
 - Allocate space to newNode
 - Assign data to newNode
 - Set newNode->next to NULL



Creating linked list of 2 node (2)

- Step 3: Create the second, ... node
 - Realign the pointers
 - `temp->next = newNode`
 - `temp = newNode`



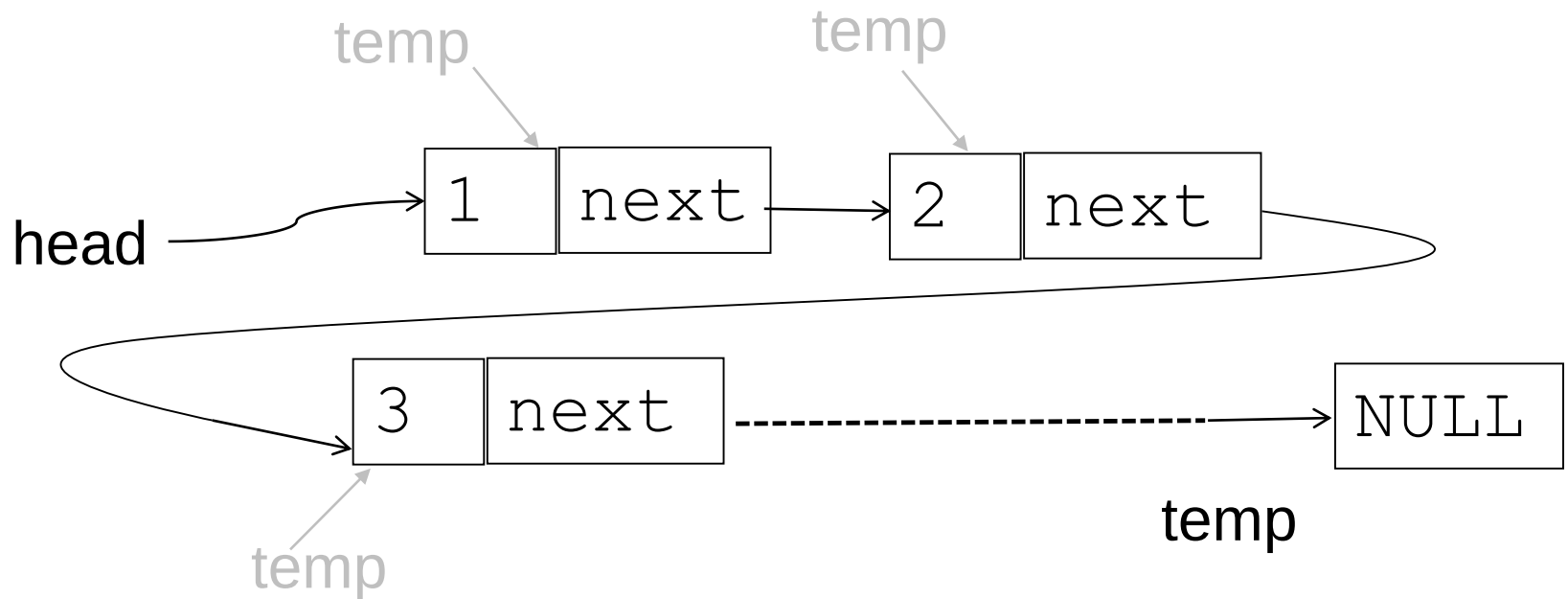


Operations on Linked List

- Creation ✓
- Traversal
- Counting the nodes
- Concatenating two lists
- Insertion of a node
- Deletion of a node

Linked list with arbitrary nodes

- Create a linked list with n number of nodes.





Creating a linked list

1. Create a self-referential structure



Creating a linked list

1. Create a self-referential structure
2. Enter the number of node



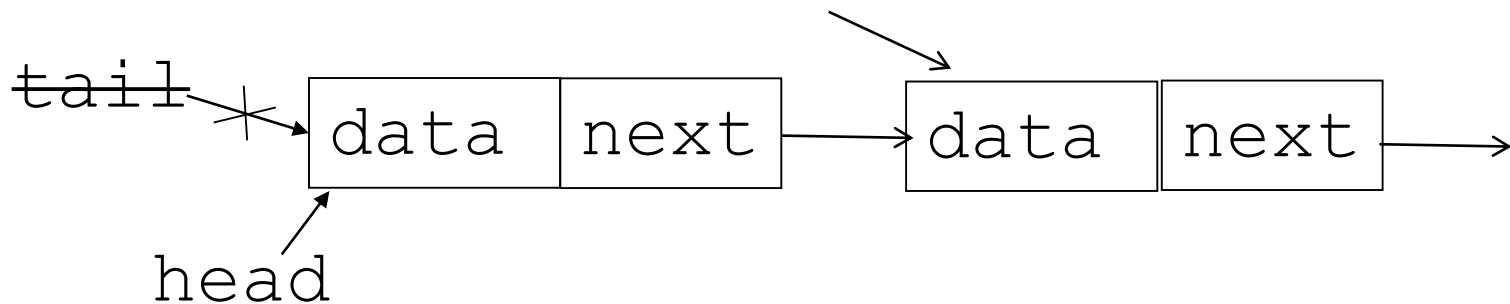
Creating a linked list

1. Create a self-referential structure
2. Enter the number of node
3. Create the first node



Creating a linked list

1. Create a self-referential structure
2. Enter the number of node
3. Create the first node
4. Create the remaining nodes





Operations on Linked List

- Creation ✓
- Traversal ✓
- Counting the nodes
- Concatenating two lists
- Insertion of a node
- Deletion of a node

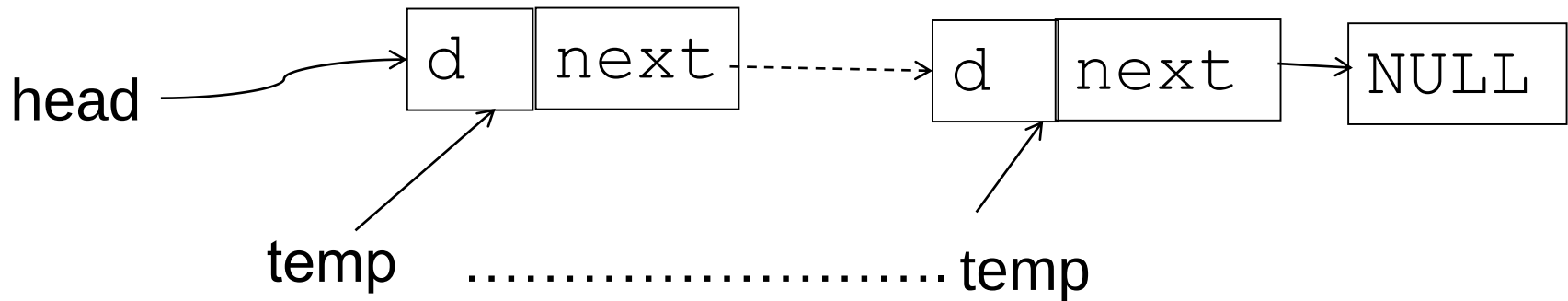


Linked List Traversal

- Traversal means “visiting” or examining each node of the list.
- Start from the beginning, visit one node at a time until the end of list.
- In order to traverse elements of a list, pointer to the first node (`head`) is all that is required.

Linked List Traversal

- Set pointer $temp$ to head
- Move the pointer ($temp$) through the list





Operations on Linked List

- Creation ✓
- Traversal ✓
- Counting the nodes ✓
- Concatenating two lists
- Insertion of a node
- Deletion of a node



Counting a linked list

- Count the number of nodes in the list.
- If the head pointer points to NULL then number of nodes present is 0.
- Else keep track of the nodes visited and traverse through the list.



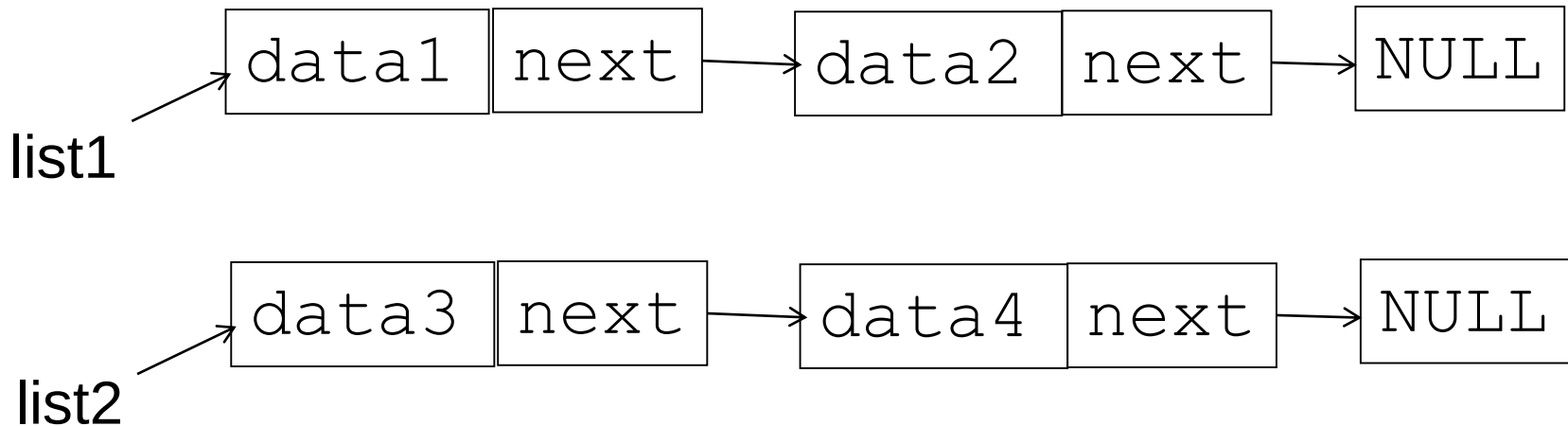
Operations on Linked List

- Creation ✓
- Traversal ✓
- Counting the nodes ✓
- Concatenating two lists ✓
- Insertion of a node
- Deletion of a node



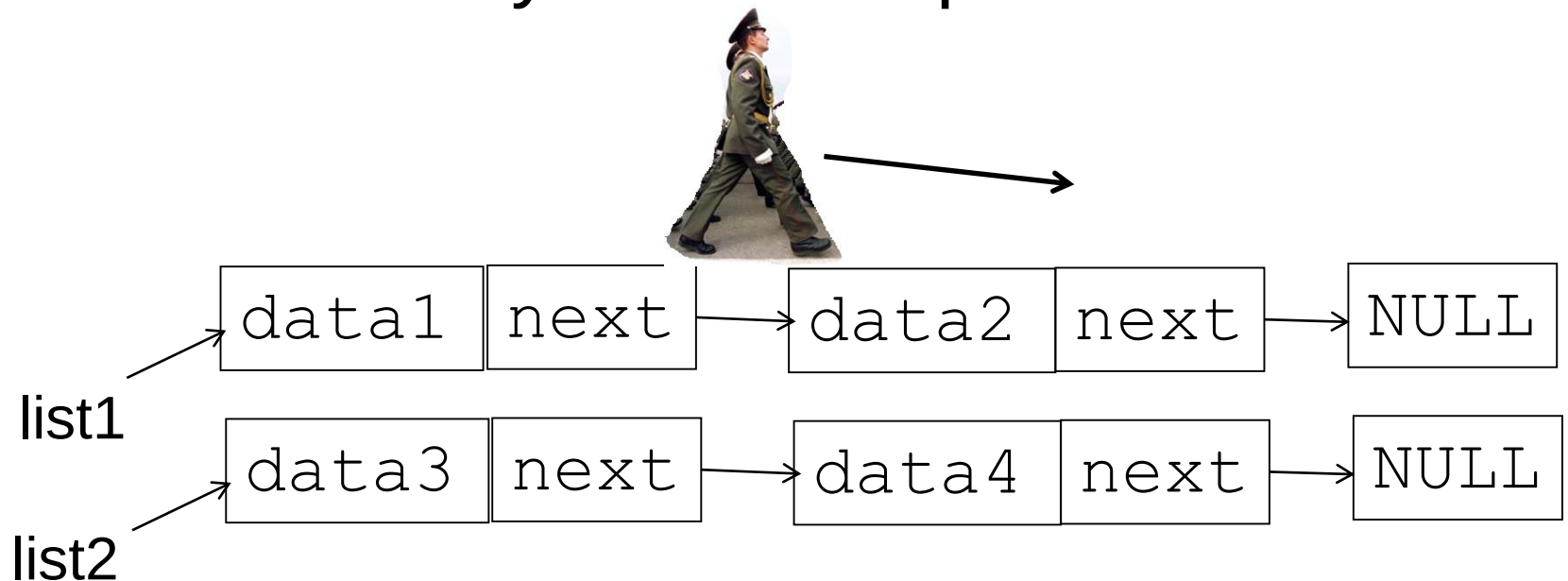
Concatenating two lists

- Take two lists *list1* and *list2* and return a single combined list.



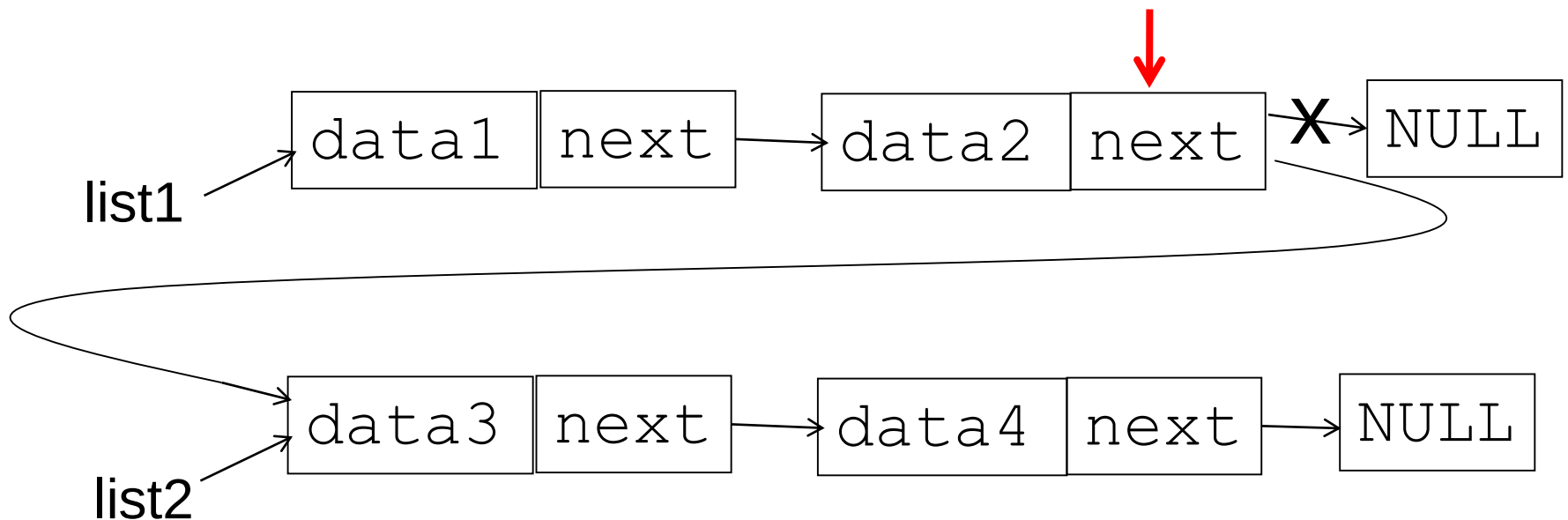
Concatenating two lists (2)

- March down list *a* looking for its end as marked by the NULL pointer.



Concatenating two lists (3)

- Keep track of the **last non NULL** pointer of *list1* and set this pointer to *list2*.





Concatenating two lists

- Steps
 - Create list1
 - Create list2
 - Concatenate list2 to list1
 - Print the list



Operations on Linked List

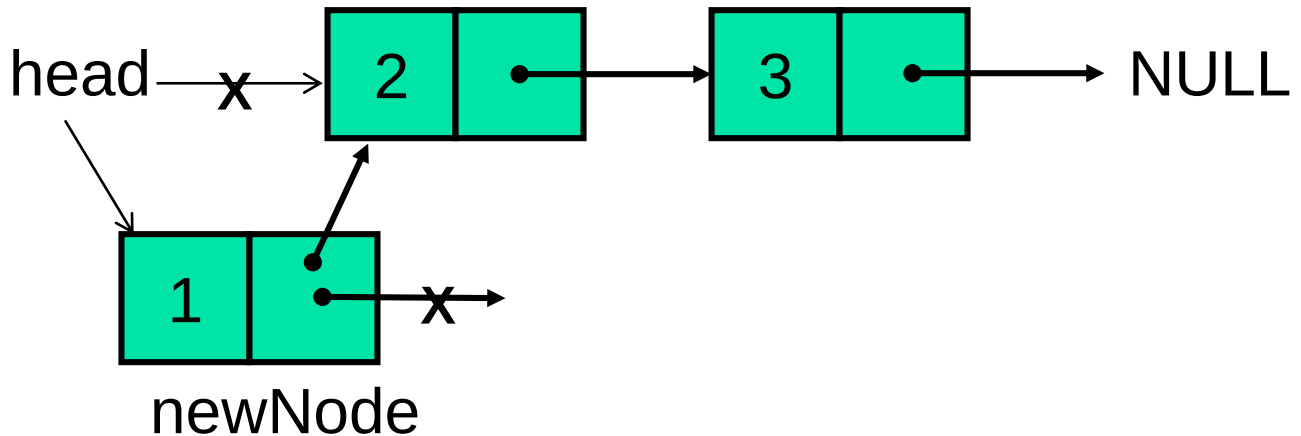
- Creation ✓
- Traversal ✓
- Counting the nodes ✓
- Concatenating two lists ✓
- Insertion of a node ✓
- Deletion of a node

Insertion in Linked List

- Where to insert a new node ?
- Two possible positions
 - Beginning of the list
 - Somewhere in the list
- Insertion takes a fixed amount of time once the position is identified.

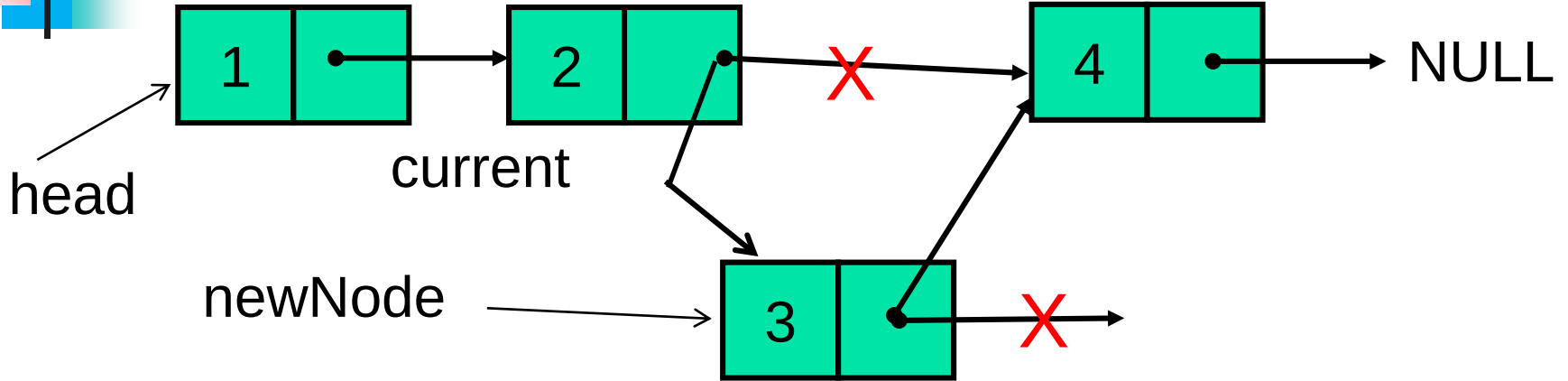


Insertion at the beginning



1. Create a new node newNode
2. Insert data to newNode
3. Let newNode->next point to head
4. Let head point to newNode

Insertion after a given node



1. Traverse the list to find the specified position - 1
2. Let $\text{newNode} \rightarrow \text{next}$ point to $\text{current} \rightarrow \text{next}$
3. Let $\text{current} \rightarrow \text{next}$ point to newNode



Operations on Linked List

- Creation ✓
- Traversal ✓
- Counting the nodes ✓
- Concatenating two lists ✓
- Insertion of a node ✓
- Deletion of a node ✓

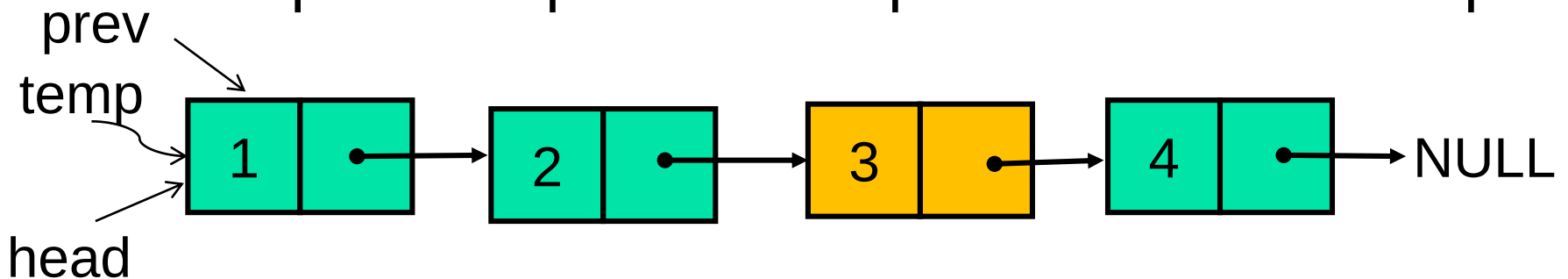


Deletion in Linked List

- Like insertion deletion takes finite amount of time if the position of the node to be deleted is identified.
- Deletion of a node can be done by making the predecessor of the node point to the successor of the node.

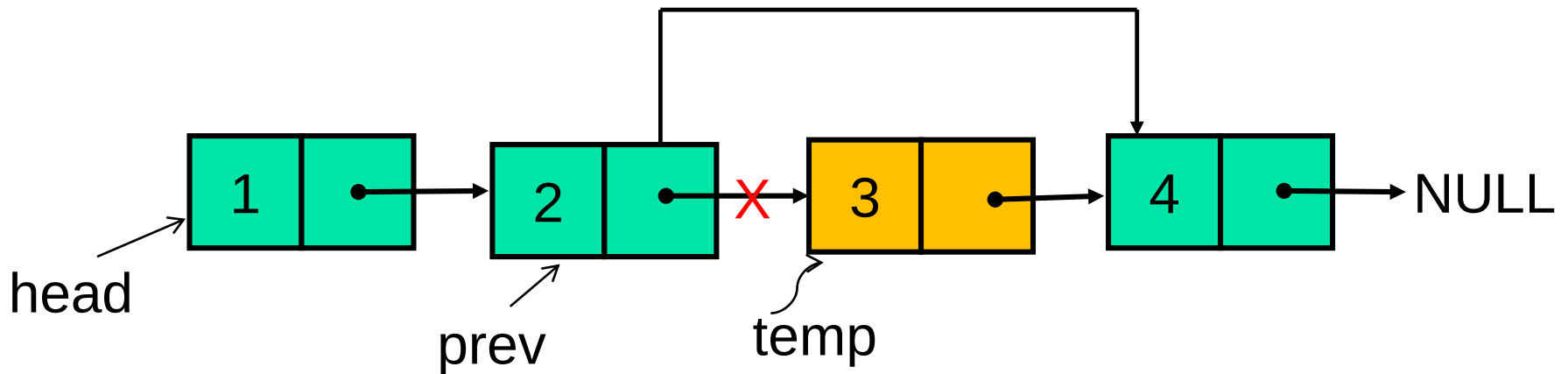
How to delete a node (1)

- Delete the node containing element 3 from the linked list
 - temp will point to the node to be deleted
 - prev will point to the previous node of tmp



How to delete a node (2)

- Set prev->next to temp->next
- Node 3 is no longer accessible, such inaccessible nodes are called garbage.
 - We want to avoid such situations.



How to delete a node (3)

- Then free temp, free(temp)
- The memory occupied by node 3 is returned to the system

